

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

Dot .Net Core and Progressive Web Application Practical

Practical – VI

Roll No.:T039	Name:Bhavesh Ramakant Salaskar
Class:TYIT	Batch:1
Date of Assignment:21-07-2026	Date/Time of Submission:28-07-2026

Steps to Convert ASP.NET MVC to PWA

1.Create the MVC Project:Step 1.

Create a new ASP.NET Core MVC project and name it html to pwa.

2.Add manifest.json:Step 2.

Right-click on the wwwroot folder in Solution Explorer, select Add > New Item, search for JSON File, and name it manifest.json.

3.Configure the Manifest File:Step 3.

Open manifest.json and add the code

4.Create the Icons Folder:Step 4.

Create a new folder inside wwwroot named icons and place your app icon images inside it.

5.Add the Service Worker File:Step 5.

Add a new JavaScript file named service-worker.js in wwwroot with the code:

6.Link Manifest in _Layout.cshtml:Step 6.

Open _Layout.cshtml (found under Views/Shared/) and add the link tag inside the <head> section:

a. Create a simple website for a College Information Portal and convert it into a Progressive Web Application.

Code:

Manifest.json:

```
{
  "name": "HTML to PWA",
  "short_name": "HTP",
  "start_url": "/",
  "display": "standalone",
  "icons": [
    {
      "src": "icons/animal (1).png",
      "sizes": "128x128",
      "type": "image/png",
      "purpose": "any"
    }
  ],
  {
    "src": "icons/animal.png",
```

```

    "sizes": "512x512",
    "type": "image/png",
    "purpose": "any"
  }
]
}

```

[service-worker.js](#)

```

self.addEventListener("install", e => {
  console.log("service worker installed");
});

```

```

self.addEventListener("fetch", e => {
  e.responseWith(fetch(e.request));
});

```

_Layout.cshtml

```

<link rel="manifest" href="/Manifest.json" />

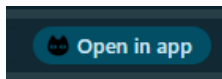
```

```

<script>
  if('serviceWorker' in navigator{
    navigator.serviceWorker.register('/service-worker.js');
  }
</script>

```

Output:



b. Create a PWA that displays a custom offline page when the user is not connected to the internet.

Code:

Offline.html:

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Offline</title>
</head>
<body>
  <h1>No internet connection</h1>
  <p> please connect to the internet and try again</p>
</body>
</html>

```

[service-worker.js](#):

```
const CACHE = "v1";
```

```
self.addEventListener("install", e => {  
  e.waitUntil(  
    caches.open(CACHE).then(cache => {  
      return cache.addAll([  
        "/",  
        "/offline.html"  
      ]);  
    })  
  );  
  console.log("Service worker installed");  
});
```

```
self.addEventListener("fetch", e => {  
  e.respondWith(  
    fetch(e.request).catch(() => caches.match("/offline.html"))  
  );  
});
```

Output:

